Sistema de Gestión de Reservas – Hotel Costa Azul S.A

Daniel Barrientos Salas

Programación Cliente/Servidor Concurrente

Análisis y Diseño

EL Hotel Costa Azul S.A: requiere una aplicación en Java que permita administrar reservas de habitaciones, controlar disponibilidad, calcular ingresos y almacenar esa información como archivos.

El sistema debe permitir registrar las reservas con información del cliente, de la habitacion, de las fechas de entrada y salida y la cantidad de personas además del precio, el cual es sistema tiene que calcular de forma automática con reglas ya predefinidas de precios en esas tarifas. El sistema tiene que realizar validaciones de los datos e impedir que se ingresen datos como cantidades de huéspedes negativas o 0, campos vacíos, fechas de salida anteriores a la fecha de ingreso, o IDs duplicados. También debe permitir poder consultar esas reservas. Como también manejar errores y toda la información guardarla en un archivo.

Identificación de las clases

1. Habitación (abstract)

Existen varios tipos de habitaciones, comparten atributos y métodos, pero su tarifa cambia así que se guarda esta como abstracta.

Atributos: tipo(String), tarifaBase(double)

Métodos: calcularTarifa(), getTipo

2. HabitaciónEstandar (Hereda de Habitación)

Método sobrescrito: calcularTatifa()

3. HabitacionEjecutiva (Hereda de Habitación)

Atributos: extraTarifa(double)

Método sobrescrito: calcularTarifa()

4. SuiteFamiliar (Hereda de Habitación)

Atributos: extraTarifa(double)

Método sobrescrito: calcularTarifa()

5. Reserva

Es la clase que representa la reserva realizada por clientes

Atributos: id (Int), nombreCompleto(String), Habitacion(Habitacion), fechaIngreso(Date), fechaSalida(Date), cantidadHuespedes(int), costoTotal(double)

Métodos: calcularCosto(), validarFechas(), validarHuespedes(), getters/setters

6. AdministradorReservas

Clase que se encargara de administrar las reservas de la empresa, como agregar, eliminar, consultar.

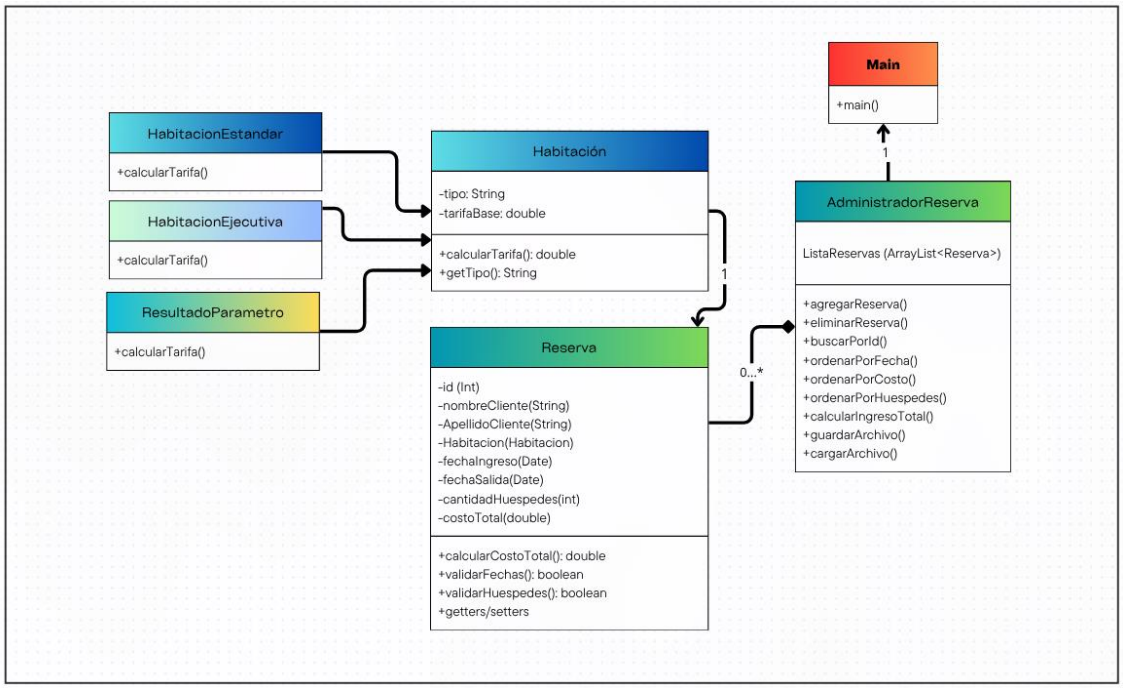
Atributos: listaReservas (ArrayList<Reserva>)

Métodos: agregarReserva(), eliminarReserva(), buscarPorId(), ordenarPorFecha(), ordenarPorCosto(), ordenarPorHuespedes(), calcularIngresoTotal(), guardarArchivo(), cargarArchivo()

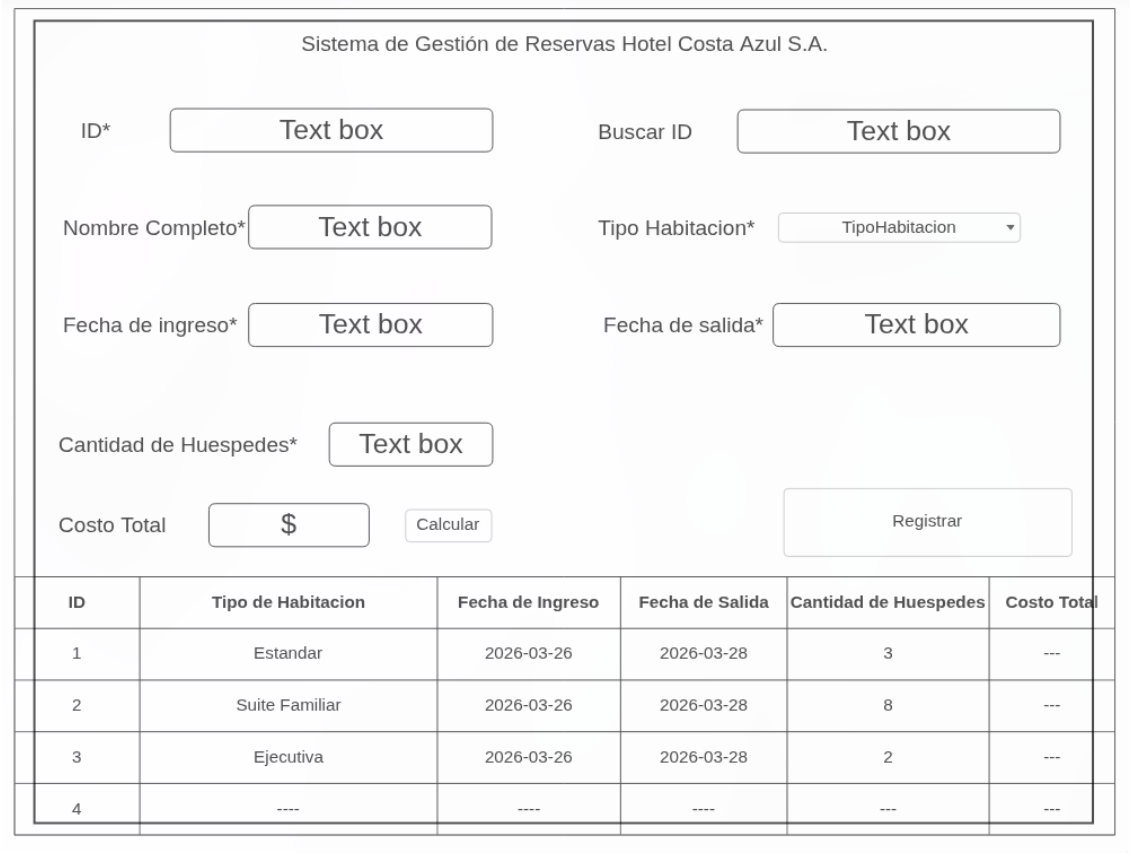
Relaciones Entre Clases

Si existe herencia y generalización porque, por ejemplo: Una Reserva tiene una Habitación lo que es Asociación, GestorReservas contiene muchas Reservas lo que lo hace Composición y una Habitación es superclase de tres tipos lo que lo hace Herencia

Diagrama UML



Mockup



ID	Tipo de Habitacion	Fecha de Ingreso	Fecha de Salida	Cantidad de Huespedes	Costo Total
1	Estandar	2026-03-26	2026-03-28	3	---
2	Suite Familiar	2026-03-26	2026-03-28	8	---
3	Ejecutiva	2026-03-26	2026-03-28	2	---
4	----	----	----	---	---

Propuesta de almacenamiento de datos

Para garantizar la conservación de la información aun en el caso de que el sistema se cierre, se implementará un mecanismo de serializacón de objetos en un archivo .bin.

Este archivo almacenará la colección completa de objetos Reserva permitiendo guardar y recuperar el estado del sistema de manera estructurada.

Flujo General del Sistema

1. Registro de una Reserva

El proceso de registro inicia pidiendole al usuario ingresar los siguientes datos:

- ID único de reserva
- Nombre completo del cliente
- Tipo de habitación
- Fecha de ingreso
- Fecha de salida
- Cantidad de huéspedes

Al presionar el botón “Registrar”, el sistema ejecuta las siguientes acciones:

1. Verifica que no existan campos vacíos.

2. Valida que la fecha de salida sea posterior a la fecha de ingreso.
3. Comprueba que la cantidad de huéspedes sea mayor a cero.
4. Verifica que el ID no esté duplicado en la colección.
5. Calcula automáticamente la tarifa por noche según el tipo de habitación.
6. Calcula el costo total multiplicando la tarifa por la cantidad de noches.
7. Crea el objeto Reserva.
8. Agrega la reserva al ArrayList<Reserva>.
9. Actualiza la tabla visual (JTable) mostrando la nueva reserva.
10. En caso de que alguna validación de falla, el sistema muestra un mensaje de error y no registra nada.

2. Consulta de Reservas

El sistema permite visualizar todas las reservas registradas mediante una tabla que se actualiza dinámicamente con la información almacenada en memoria.

Cada fila de la tabla representa una reserva y muestra: ID, Nombre del cliente, Tipo de habitación, Fechas, Cantidad de huéspedes, Costo total

Esta visualización permite al usuario analizar rápidamente la ocupación y la información registrada.

3. Ordenamiento de la información

El sistema permite ordenar las reservas según distintos criterios:

- Fecha de ingreso
- Costo total
- Cantidad de huéspedes

Cuando el usuario selecciona un criterio de ordenamiento:

1. Se aplica un Comparator correspondiente.
2. Se reorganiza el ArrayList<Reserva>.
3. Se actualiza la tabla para reflejar el nuevo orden.

4. Calculo de ingreso Total

El sistema incluye una función que recorre la lista de reservas y suma el costo total de cada una. Este valor representa el ingreso acumulado proyectado del hotel y se muestra al usuario mediante un cuadro de diálogo

5. Almacenamiento de Información

Para mantener los datos aun cuando la aplicación se cierre, el sistema permite guardar la información en un archivo (reservas.bin). Lo que realiza es:

- Serializar el ArrayList<Reserva>.
- Escribirlo en el archivo mediante ObjectOutputStream.

6. Recuperación de Información

Al iniciar el sistema:

- Se verifica la existencia del archivo.
- Se deserializa el contenido del archivo binario.
- Se reconstruye la colección en memoria.
- Se actualiza la tabla con la información cargada.